Homework 3: Variogram models and kriging

Spatial statistics 2026–2027 · Professor: Dr. Lasinio

Mojtaba Alinejad

28 October 2026

Contents

1 Data and exploratory checks **1**
1.1 What I am measuring . . . 1
1.2 Trend and distribution . . . 3
1.3 Raw and residual variograms . . . 4

2 Choosing a variogram model by likelihood **7**
2.1 Model and comparison rule . . . 7
2.2 Selected parameters and fitted curve . . . 8

3 Kriging with the likelihood estimates **11**
3.1 What I predict . . . 11
3.2 A short check at held-out stations . . . 11
3.3 Final predictions on a 20 by 20 grid . . . 13

4 My AI prompt and assessment **15**

5 What I learned **15**

In this homework, I return to the Wolfcamp data. I choose a variogram model, estimate its parameters by maximum likelihood, and use it for kriging. I first check the spatial trend, because a changing mean can affect the raw variogram. I also make a map of prediction uncertainty and check predictions at stations left out of the fitting process.

1 Data and exploratory checks

1.1 What I am measuring

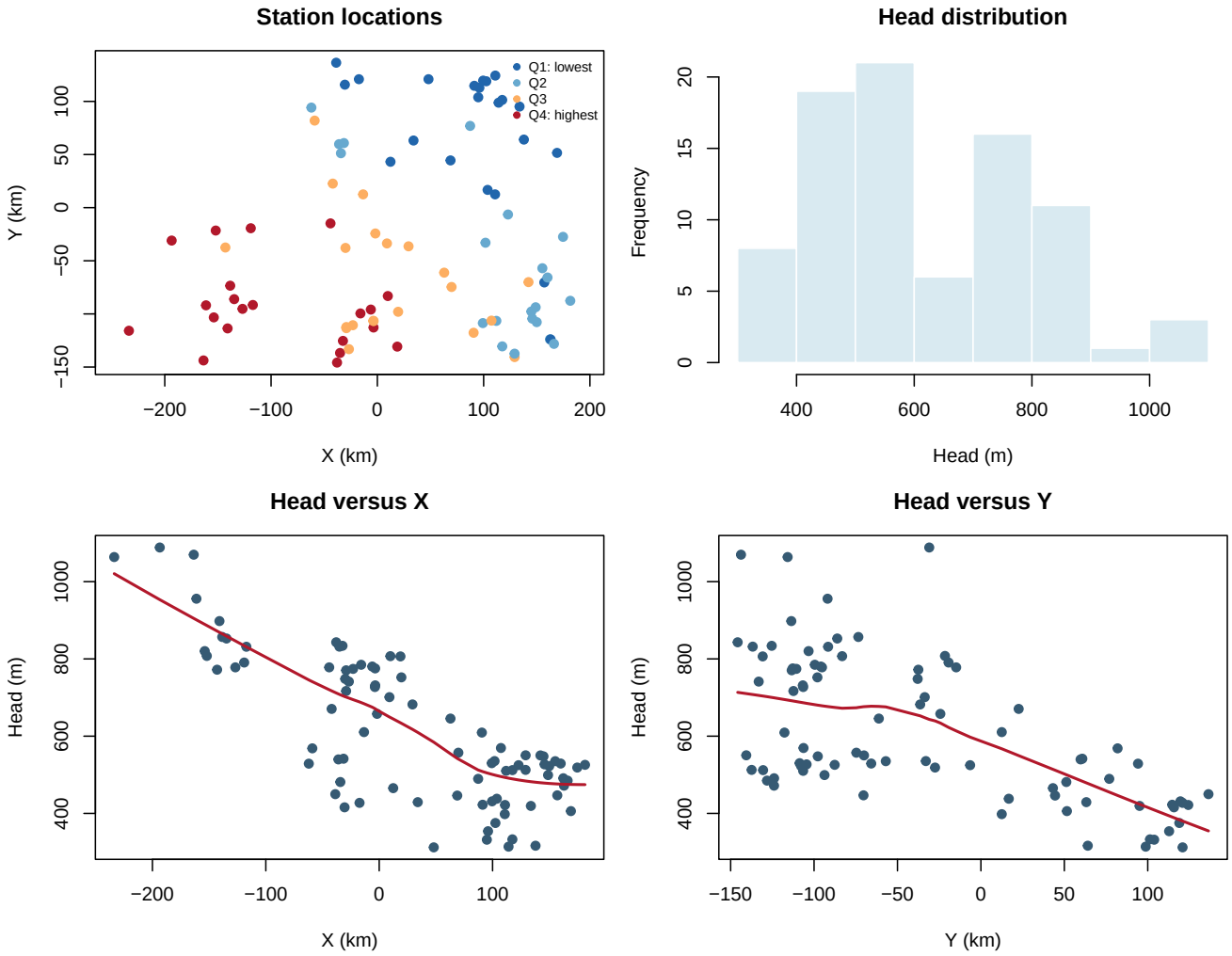
Wolfcamp contains piezometric-head measurements from 85 locations in an aquifer in Texas. The `geoR` documentation states that coordinates are in **kilometres** and heads are in **metres**. These are continuous measurements at spatial locations. I use the supplied projected coordinates for distances.

```
data("wolfcamp", package = "geoR")
dat <- data.frame(x = wolfcamp$coords[, 1],
                  y = wolfcamp$coords[, 2], head = wolfcamp$data)
checks <- data.frame(
  Stations = nrow(dat), Missing = sum(is.na(dat)),
  Repeated_locations = sum(duplicated(dat[c("x", "y")])),
  Minimum_head_m = min(dat$head), Maximum_head_m = max(dat$head)
)
stopifnot(all(is.finite(as.matrix(dat))),
          checks$Repeated_locations == 0)
knitr::kable(checks, digits = 2)
```

Stations	Missing	Repeated_locations	Minimum_head_m	Maximum_head_m
85	0	0	312.11	1088.42

I keep all 85 observations. There are no missing values or repeated locations. I do not treat a high or low head as an error just because it is different from nearby values.

```
quartile <- cut(dat$head, quantile(dat$head, seq(0, 1, 0.25)),
                include.lowest = TRUE, labels = FALSE)
station_col <- c("#2166AC", "#67A9CF", "#FDAE61", "#B2182B")
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
plot(dat$x, dat$y, asp = 1, pch = 19, col = station_col[quartile],
     xlab = "X (km)", ylab = "Y (km)", main = "Station locations")
legend("topright", c("Q1: lowest", "Q2", "Q3", "Q4: highest"),
       col = station_col, pch = 19, bty = "n", cex = 0.7)
hist(dat$head, breaks = "FD", col = "#D9EAF1", border = "white",
     main = "Head distribution", xlab = "Head (m)")
plot(dat$x, dat$head, pch = 19, col = "#365A73",
     xlab = "X (km)", ylab = "Head (m)", main = "Head versus X")
lines(lowess(dat$x, dat$head), col = "#B2182B", lwd = 2)
plot(dat$y, dat$head, pch = 19, col = "#365A73",
     xlab = "Y (km)", ylab = "Head (m)", main = "Head versus Y")
lines(lowess(dat$y, dat$head), col = "#B2182B", lwd = 2)
```



I see higher heads towards the southwest and lower heads towards the northeast. The sites are unevenly spaced. This broad pattern makes a constant mean questionable, so I examine trends before choosing the covariance model.

1.2 Trend and distribution

I start with ordinary linear regressions as exploratory checks. I compare a constant mean, a linear coordinate trend and a quadratic coordinate trend. Later, the likelihood models estimate the trend while allowing spatially correlated errors; I do not use these ordinary regression fits as the final kriging model.

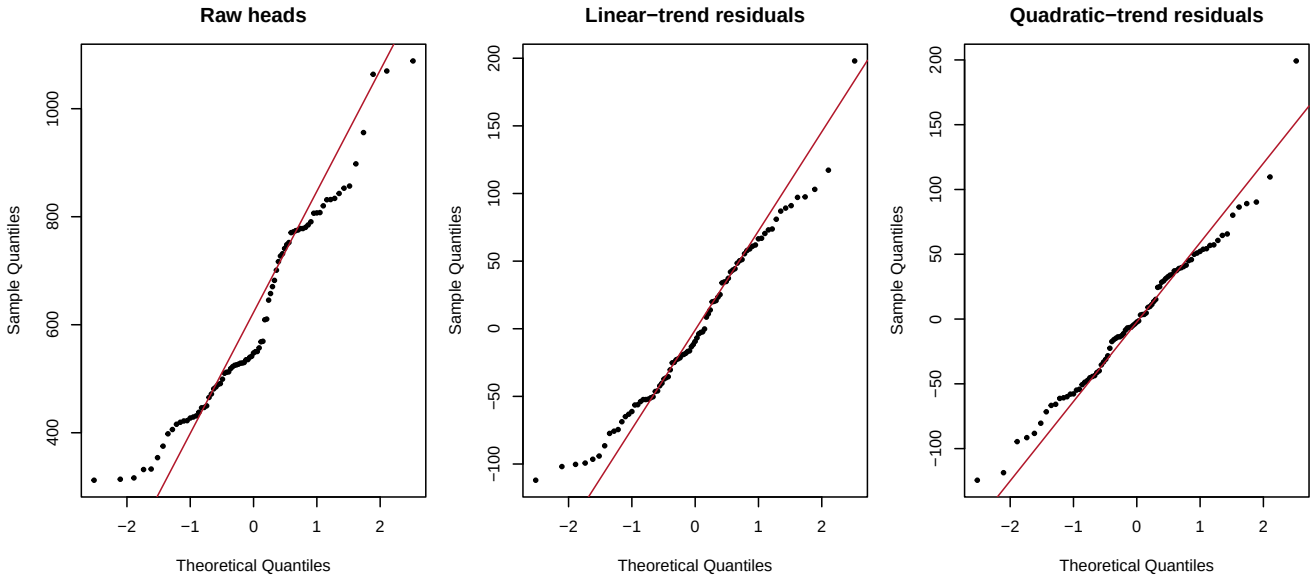
```
ols <- list(
  Constant = lm(head ~ 1, data = dat),
  Linear = lm(head ~ x + y, data = dat),
  Quadratic = lm(head ~ x + y + I(x^2) + I(y^2) + I(x*y), data = dat)
)
trend_check <- data.frame(
  Trend = names(ols),
  Adjusted_R2 = sapply(ols, function(m) summary(m)$adj.r.squared),
  Residual_SD_m = sapply(ols, function(m) sd(residuals(m)))
)
knitr::kable(trend_check, digits = 3, row.names = FALSE)
```

Trend	Adjusted_R2	Residual_SD_m
Constant	0.000	186.333

Trend	Adjusted_R2	Residual_SD_m
Linear	0.888	61.548
Quadratic	0.906	55.301

The linear trend explains much of the broad pattern: its adjusted R-squared is **0.888**. A quadratic trend improves the ordinary regression fit, but it adds three mean coefficients. I do not choose it from R-squared alone. I compare the trends again using the spatial likelihood and a penalty for the number of parameters.

```
par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
qqnorm(dat$head, main = "Raw heads", pch = 19, cex = 0.55)
qqline(dat$head, col = "#B2182B")
qqnorm(residuals(ols$Linear), main = "Linear-trend residuals",
       pch = 19, cex = 0.55)
qqline(residuals(ols$Linear), col = "#B2182B")
qqnorm(residuals(ols$Quadratic), main = "Quadratic-trend residuals",
       pch = 19, cex = 0.55)
qqline(residuals(ols$Quadratic), col = "#B2182B")
```



The residuals look closer to a straight Q-Q line than the raw heads, although there are some differences in the tails. I keep the original metre scale and use a Gaussian residual process as a working model. A normal-looking marginal distribution does not prove that the joint spatial distribution is Gaussian. I therefore check the predictive errors as well. No transformation is used in the fitting, validation or mapping: `lambda = 1` throughout.

1.3 Raw and residual variograms

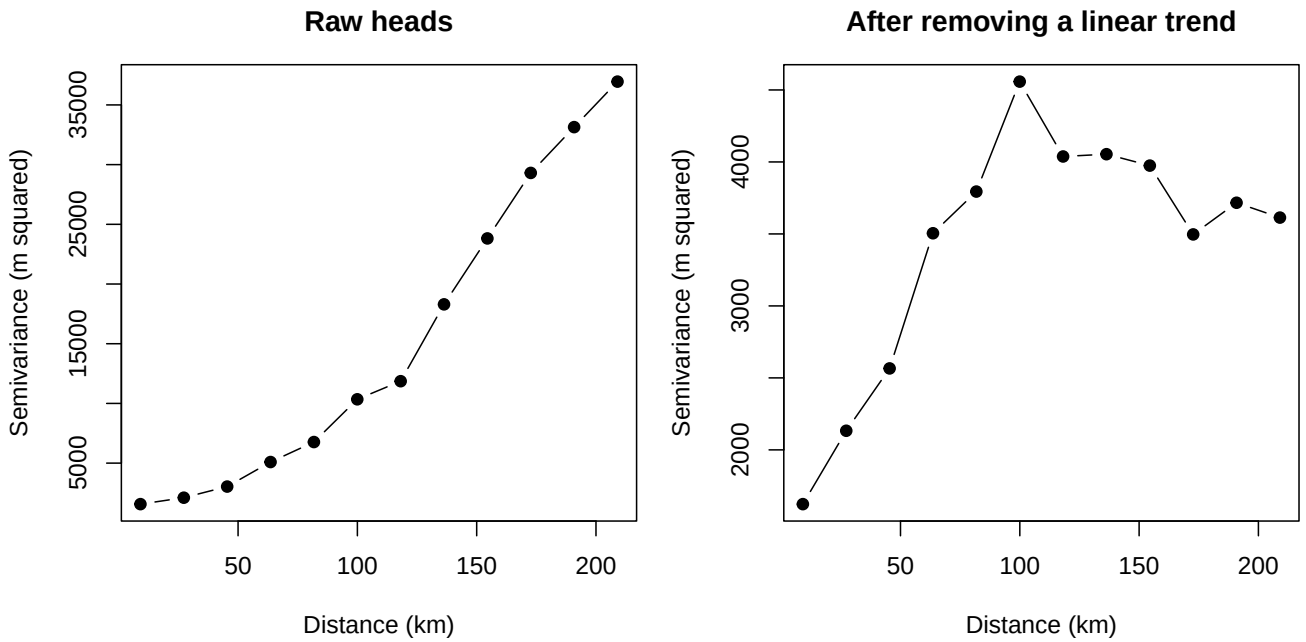
I use 12 equal-width distance groups, stopping at half the maximum station separation. I show the pair counts so that the amount of information in each group is visible. The residual variogram below removes a linear trend by ordinary least squares, as documented for `geoR::variog`.

```
cutoff <- max(dist(wolfcamp$coords)) / 2
breaks <- seq(0, cutoff, length.out = 13)
v_raw <- geoR::variog(wolfcamp, breaks = breaks, trend = "cte",
                      lambda = 1, messages = FALSE)
v_linear <- geoR::variog(wolfcamp, breaks = breaks, trend = "1st",
                         lambda = 1, messages = FALSE)
knitr::kable(data.frame(
```

```
  Lag_centre_km = v_linear$u, Pairs = v_linear$n,
  Raw_semivariance = v_raw$v, Residual_semivariance = v_linear$v
), digits = 2)
```

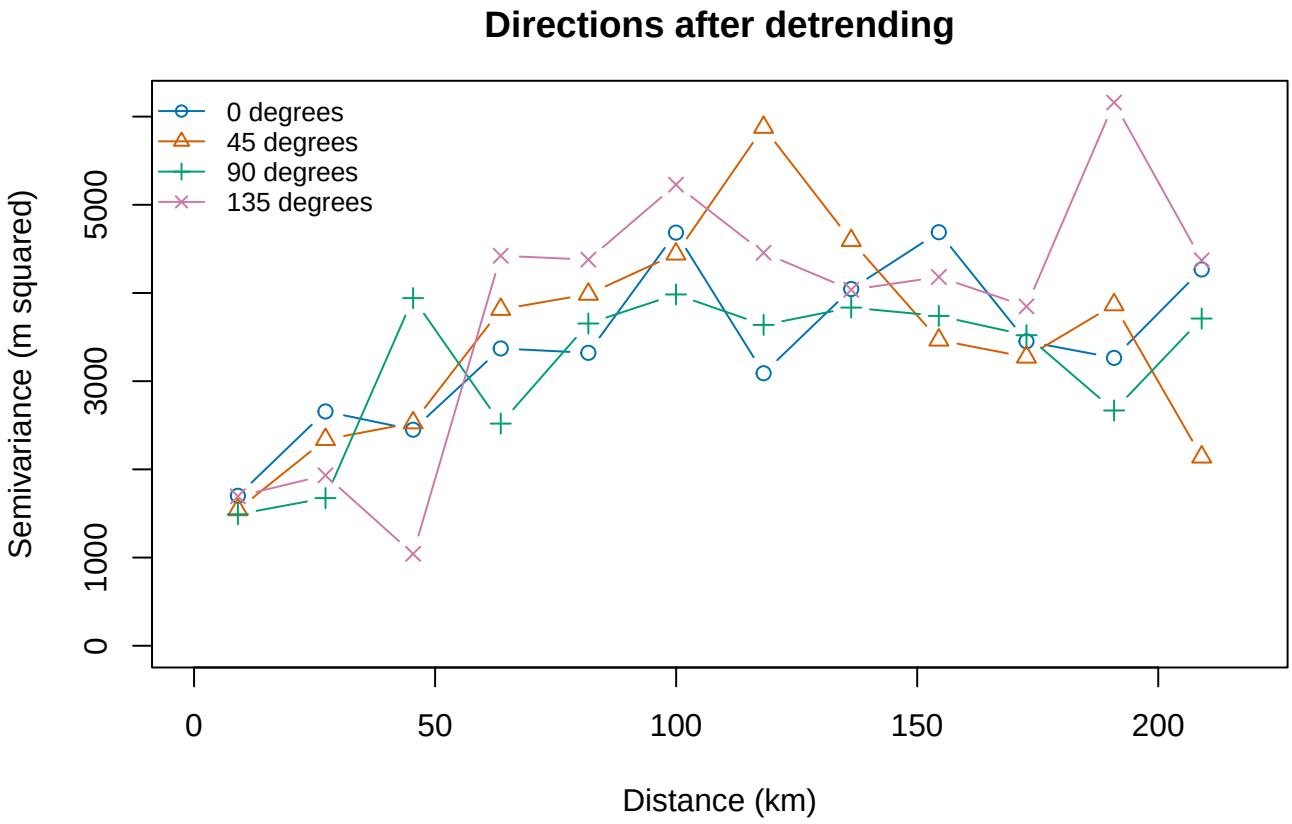
Lag_centre_km	Pairs	Raw_semivariance	Residual_semivariance
9.09	70	1563.74	1623.09
27.26	131	2104.34	2132.78
45.44	160	3034.78	2565.87
63.61	140	5090.78	3504.60
81.79	181	6764.60	3794.23
99.96	193	10343.65	4558.02
118.14	214	11860.59	4037.92
136.31	248	18296.54	4053.81
154.49	310	23818.00	3974.52
172.67	310	29305.76	3496.56
190.84	294	33140.75	3716.57
209.02	207	36953.13	3613.49

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(v_raw$u, v_raw$v, type = "b", pch = 19,
     xlab = "Distance (km)", ylab = "Semivariance (m squared)",
     main = "Raw heads")
plot(v_linear$u, v_linear$v, type = "b", pch = 19,
     xlab = "Distance (km)", ylab = "Semivariance (m squared)",
     main = "After removing a linear trend")
```



The raw variogram keeps increasing over the displayed distances. After removing the trend, the semivariance rises over shorter distances and then roughly levels off. The panels have different vertical scales because the trend removes a large part of the variation. This supports modelling a changing mean plus a stationary residual process. It does not prove stationarity, and the residuals still show spatial structure.

```
angles <- c(0, 45, 90, 135)
directional <- lapply(angles, function(a) {
  geoR::variog(wolfcamp, breaks = breaks, trend = "1st",
    direction = a*pi/180, tolerance = pi/8, pairs.min = 10,
    lambda = 1, messages = FALSE)
})
cols <- c("#0072B2", "#D55E00", "#009E73", "#CC79A7")
par(mfrow = c(1, 1), mar = c(4, 4, 3, 1))
plot(c(0, cutoff), c(0, max(unlist(lapply(directional, function(v) v$v)))),
     type = "n", xlab = "Distance (km)",
     ylab = "Semivariance (m squared)", main = "Directions after detrending")
for (j in seq_along(angles)) {
  lines(directional[[j]]$u, directional[[j]]$v,
        type = "b", col = cols[j], pch = j)
}
legend("topleft", paste(angles, "degrees"), col = cols,
       pch = 1:4, lty = 1, bty = "n", cex = 0.8)
```



I use a tolerance of 22.5 degrees and show only directional bins with at least 10 pairs. The curves differ, especially at longer distances, but I do not see a simple, stable separation of ranges across directions. I keep an isotropic covariance model, which uses distance rather than direction. With only 85 sites, this remains a simplifying assumption; these curves do not rule out anisotropy.

2 Choosing a variogram model by likelihood

2.1 Model and comparison rule

My model is

$$Z(s) = m(s) + S(s) + \epsilon(s), \qquad \mathrm{Cov}\{Z(s_i), Z(s_j)\} = \sigma^2 \rho(\|s_i - s_j\|) + \tau^2 \mathbf{1}(i = j).$$

Here, `m(s)` is the trend, `S(s)` is the spatial residual process, and the independent nugget term has variance `tau^2`. The partial sill is `sigma^2`; the total sill is `sigma^2 + tau^2`. For a positive distance, the corresponding semivariogram is `tau^2 + sigma^2 * (1 - rho(h))`. At exactly zero distance, the semivariogram is zero; the nugget is its jump just to the right of zero.

I compare exponential, spherical and two Matérn models, with smoothness fixed at 1.5 and 2.5. A Matérn model with smoothness 0.5 is the exponential model under the same range convention, so the exponential candidate already covers that case. I combine these four covariance choices with the three trends.

I fit all 12 models to the original observations using **maximum likelihood (ML)** with `geoR::likfit`. I do not estimate them by fitting a curve to the empirical variogram. I use ML rather than REML for this comparison because the models have different mean structures. Their AIC values are based on the same observations and response scale.

My selection rule is to keep models within **2 AIC units of the minimum**, then choose the one with the fewest estimated parameters. If more than one remains, I choose the lower AIC. The two-unit rule is a practical way to favour simplicity when support is close, not a significance test. I also check the fitted curve, nugget and numerical convergence before accepting the result.

```
candidates <- expand.grid(
  trend = c("cte", "1st", "2nd"),
  family = c("Exponential", "Spherical", "Matern 1.5", "Matern 2.5"),
  stringsAsFactors = FALSE
)
candidates$model <- rep(c("exponential", "spherical", "matern", "matern"),
                        each = 3)
candidates$kappa <- rep(c(0.5, 0.5, 1.5, 2.5), each = 3)

fit_model <- function(g, trend, model, kappa) {
  X <- geoR::trend.spatial(trend, g)
  s2 <- mean(lm.fit(X, g$data)$residuals^2)
  # I repeat optimization from three initial range values.
  starts <- lapply(c(15, 60, 180), function(start) {
    geoR::likfit(g, trend = trend, cov.model = model,
      kappa = kappa, fix.kappa = TRUE,
      ini.cov.pars = c(0.8*s2, start), nugget = 0.2*s2,
      fix.nugget = FALSE, lambda = 1, fix.lambda = TRUE,
      lik.method = "ML", limits = geoR::pars.limits(phi = c(0.1, 2500)),
      messages = FALSE, control = list(maxit = 2000))
  })
  ok <- vapply(starts, function(m) {
    m$info.minimisation.function$convergence == 0 && is.finite(m$loglik)
  }, logical(1))
  if (!any(ok)) stop("No converged fit for this candidate.")
  valid <- starts[ok]
  best <- valid[[which.max(vapply(valid, function(m) m$loglik, numeric(1)))]]
  attr(best, "start_loglik") <-
    vapply(starts, function(m) m$loglik, numeric(1))
  attr(best, "converged_starts") <- sum(ok)
  best
```

```
}

select_model <- function(g) {
  fits <- lapply(seq_len(nrow(candidates)), function(j) {
    fit_model(g, candidates$trend[j], candidates$model[j], candidates$kappa[j])
  })
  tab <- candidates[c("trend", "family")]
  tab$K <- vapply(fits, function(m) m$npars, numeric(1))
  tab$AIC <- vapply(fits, function(m) m$AIC, numeric(1))
  tab$Delta <- tab$AIC - min(tab$AIC)
  tab$Nugget <- vapply(fits, function(m) m$nugget, numeric(1))
  tab$Partial_sill <- vapply(fits, function(m) m$cov.pars[1], numeric(1))
  tab$Phi <- vapply(fits, function(m) m$cov.pars[2], numeric(1))
  close <- which(tab$Delta <= 2)
  chosen <- close[order(tab$K[close], tab$AIC[close])][1]
  list(fit = fits[[chosen]], row = chosen, table = tab, fits = fits)
}
selection <- select_model(wolfcamp)
chosen <- selection$fit
chosen_trend <- candidates$trend[selection$row]
chosen_family <- candidates$family[selection$row]
knitr::kable(selection$table[c("trend", "family", "K", "AIC", "Delta")],
             digits = 2, row.names = FALSE)
```

trend	family	K	AIC	Delta
cte	Exponential	4	945.89	18.03
1st	Exponential	6	929.73	1.87
2nd	Exponential	9	927.87	0.00
cte	Spherical	4	944.09	16.23
1st	Spherical	6	928.73	0.87
2nd	Spherical	9	928.16	0.29
cte	Matern 1.5	4	943.73	15.86
1st	Matern 1.5	6	930.76	2.89
2nd	Matern 1.5	9	928.76	0.90
cte	Matern 2.5	4	945.07	17.21
1st	Matern 2.5	6	930.99	3.12
2nd	Matern 2.5	9	929.06	1.19

In the table, `cte` means a constant mean, `1st` a linear trend, and `2nd` a quadratic trend. `K` includes the estimated mean coefficients and the three covariance parameters. Matérn smoothness is fixed, not estimated.

The strict AIC minimum is the **quadratic-trend exponential model**, with AIC **927.87**. My rule selects the **linear-trend spherical model**, with AIC **928.73** and a difference of **0.87**. It estimates six parameters instead of nine. I accept this small AIC difference to keep the mean model simpler. I do not claim that the spherical model is the only reasonable choice: several candidates have similar support.

All constant-mean candidates have clearly higher AIC. Their fitted ranges can also become much larger than the sampled area, which is a warning that the covariance is trying to describe the broad trend. This is why I keep a trend in the final model.

2.2 Selected parameters and fitted curve

```
parameter_table <- data.frame(
  Parameter = c("Nugget (m squared)", "Partial sill (m squared)",
                "Total sill (m squared)", "Spherical range (km)",
```

```
                "Nugget / total sill"),
  Estimate = c(chosen$nugget, chosen$cov.pars[1],
               chosen$nugget + chosen$cov.pars[1], chosen$cov.pars[2],
               chosen$nugget / (chosen$nugget + chosen$cov.pars[1]))
)
knitr::kable(parameter_table, digits = 3)
```

Parameter	Estimate
Nugget (m squared)	1236.242
Partial sill (m squared)	3328.955
Total sill (m squared)	4565.197
Spherical range (km)	122.948
Nugget / total sill	0.271

```
knitr::kable(data.frame(Coefficient = names(chosen$beta),
                        Estimate = as.numeric(chosen$beta)), digits = 3)
```

Coefficient	Estimate
intercept	620.355
x	-1.326
y	-1.206

```
knitr::kable(data.frame(
  Initial_range_km = c(15, 60, 180),
  Final_log_likelihood = attr(chosen, "start_loglik")
), digits = 4)
```

Initial_range_km	Final_log_likelihood
15	-460.5162
60	-458.3671
180	-458.3671

```
stopifnot(chosen$info.minimisation.function$convergence == 0,
          chosen$cov.pars[2] > 0.1, chosen$cov.pars[2] < 2500,
          chosen$nugget >= 0, chosen$cov.pars[1] > 0)
```

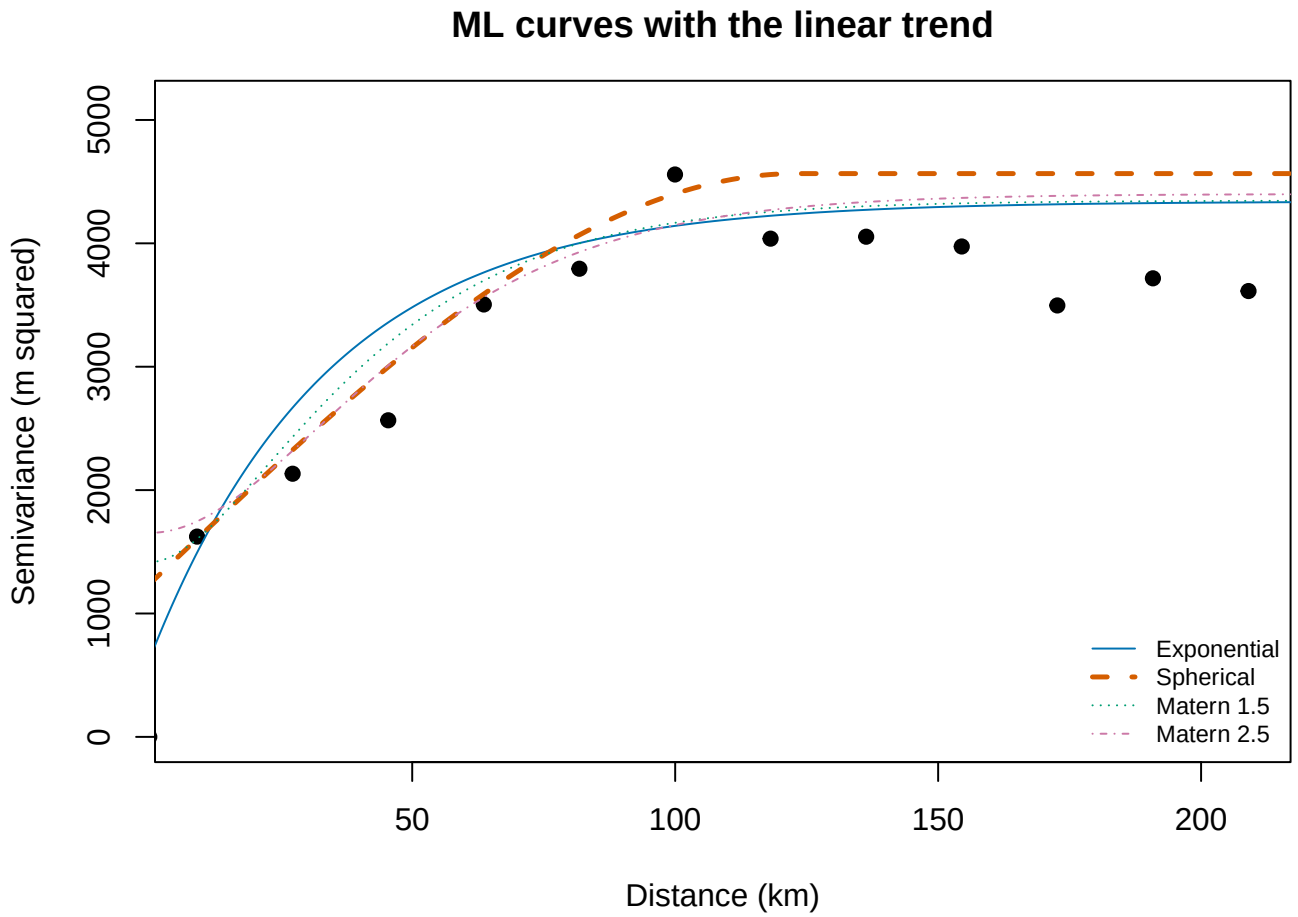
For the spherical model, correlation reaches zero at the fitted range, **122.9 km**. This is different from an exponential model, whose correlation approaches zero gradually. I therefore do not compare the `phi` values across covariance families as if they had the same meaning.

The nugget is about **27.1%** of the total sill. It can include measurement error and variation at distances smaller than those resolved by the stations. These data alone do not separate those sources. I do not choose a model just because it has the smallest nugget; the likelihood score, mean structure and covariance shape matter together.

The three starting values do not all reach the same likelihood. Two reach the better solution, so using multiple starts matters here. I keep the converged fit with the greatest likelihood. The allowed range interval, 0.1 to 2,500 km, is a numerical bound; the selected estimate lies well inside it. These checks reduce the risk of accepting a poor numerical fit but do not prove that the global optimum has been found.

```
lag <- seq(0.001, cutoff, length.out = 300)
same_trend <- which(candidates$trend == chosen_trend)
model_cols <- c("#0072B2", "#D55E00", "#009E73", "#CC79A7")
par(mfrow = c(1, 1), mar = c(4, 4, 3, 1))
```

```r
plot(v_linear$u, v_linear$v, pch = 19,
     ylim = c(0, 1.12*max(v_linear$v, chosen$nugget + chosen$cov.pars[1])),
     xlab = "Distance (km)", ylab = "Semivariance (m squared)",
     main = "ML curves with the linear trend")
for (j in seq_along(same_trend)) {
  m <- selection$fits[[same_trend[j]]]
  gamma <- m$nugget + m$cov.pars[1] -
    geoR::cov.spatial(lag, cov.model = m$cov.model,
                      cov.pars = m$cov.pars, kappa = m$kappa)
  lines(lag, gamma, col = model_cols[j],
        lwd = if (same_trend[j] == selection$row) 2.5 else 1,
        lty = j)
}
points(0, 0, pch = 1)
legend("bottomright", candidates$family[same_trend],
       col = model_cols, lty = 1:4, lwd = c(1, 2.5, 1, 1),
       bty = "n", cex = 0.75)
```



The fitted spherical curve follows the rise over shorter distances and then levels off. It does not pass through every empirical point, especially at longer lags. The empirical plot uses an ordinary regression trend, while ML estimates the mean and covariance together from the individual observations. I use the plot as a check, not as a requirement that the fitted curve match every bin. The nearby curves also show why I should be cautious about declaring one covariance family clearly superior.

3 Kriging with the likelihood estimates

3.1 What I predict

I use the selected ML estimates in **universal kriging** because the mean contains a linear coordinate trend. In `geoR`, this is specified with `type.krige = "OK"` and the same nonconstant trend at the observation and prediction locations. Despite that argument name, the nonconstant trend makes this universal rather than constant-mean ordinary kriging.

For the main map, I predict the smooth underlying head, `m(s) + S(s)`, excluding the independent nugget term. I set `signal = TRUE` and `micro.scale = 0` explicitly. This treats all the nugget as variation excluded from that smooth target; it does not establish that the nugget is entirely instrument error.

```
predict_head <- function(g, locations, fit, trend, signal = TRUE) {
  geoR::krige.conv(g, locations = as.matrix(locations),
    krige = geoR::krige.control(type.krige = "OK", obj.model = fit,
      trend.d = trend, trend.l = trend, micro.scale = 0),
    output = geoR::output.control(signal = signal, messages = FALSE))
}
```

The kriging variance allows for estimating the trend coefficients, while treating the fitted covariance parameters as fixed. It therefore does not include all the uncertainty from estimating the range, sill, nugget or choosing the model. This is likelihood-based prediction with fitted parameters, not Bayesian prediction averaged over a posterior distribution.

3.2 A short check at held-out stations

I use five random folds, with 17 stations in each fold. In each training set, I repeat the whole AIC comparison and selection rule, then predict the held-out stations. I do not use their heads to choose the trend or covariance model. This checks the model-selection procedure rather than treating a model chosen on all the observations as if it had been fixed in advance.

For this check, the target is an observed head at a new location. I therefore set `signal = FALSE`, so the variance includes the nugget. The smooth-map target and the observed-validation target have different prediction variances.

```
set.seed(2026)
fold <- sample(rep(1:5, length.out = nrow(dat)))
cv_pred <- cv_var <- numeric(nrow(dat))
fold_rows <- vector("list", 5)
for (k in 1:5) {
  test <- fold == k
  train <- geoR::as.geodata(dat[!test, ], coords.col = 1:2, data.col = 3)
  inside <- select_model(train)
  trend <- candidates$trend[inside$row]
  pred <- predict_head(train, dat[test, c("x", "y")],
                       inside$fit, trend, signal = FALSE)
  cv_pred[test] <- pred$predict
  cv_var[test] <- pred$krige.var
  fold_rows[[k]] <- data.frame(
    Fold = k, Trend = trend, Family = candidates$family[inside$row],
    RMSE_m = sqrt(mean((cv_pred[test] - dat$head[test])^2))
  )
}
stopifnot(all(is.finite(cv_pred)), all(is.finite(cv_var)), all(cv_var > 0))
errors <- cv_pred - dat$head
cv_scores <- data.frame(
  RMSE_m = sqrt(mean(errors^2)), MAE_m = mean(abs(errors)),
  Mean_error_m = mean(errors), MSSE = mean(errors^2 / cv_var),
  Coverage_95_percent = 100*mean(abs(errors) <= qnorm(0.975)*sqrt(cv_var))
```

```
)
knitr::kable(do.call(rbind, fold_rows), digits = 2, row.names = FALSE)
```

Fold	Trend	Family	RMSE_m
1	1st	Spherical	58.89
2	2nd	Exponential	44.37
3	2nd	Exponential	44.71
4	1st	Spherical	72.43
5	1st	Exponential	50.91

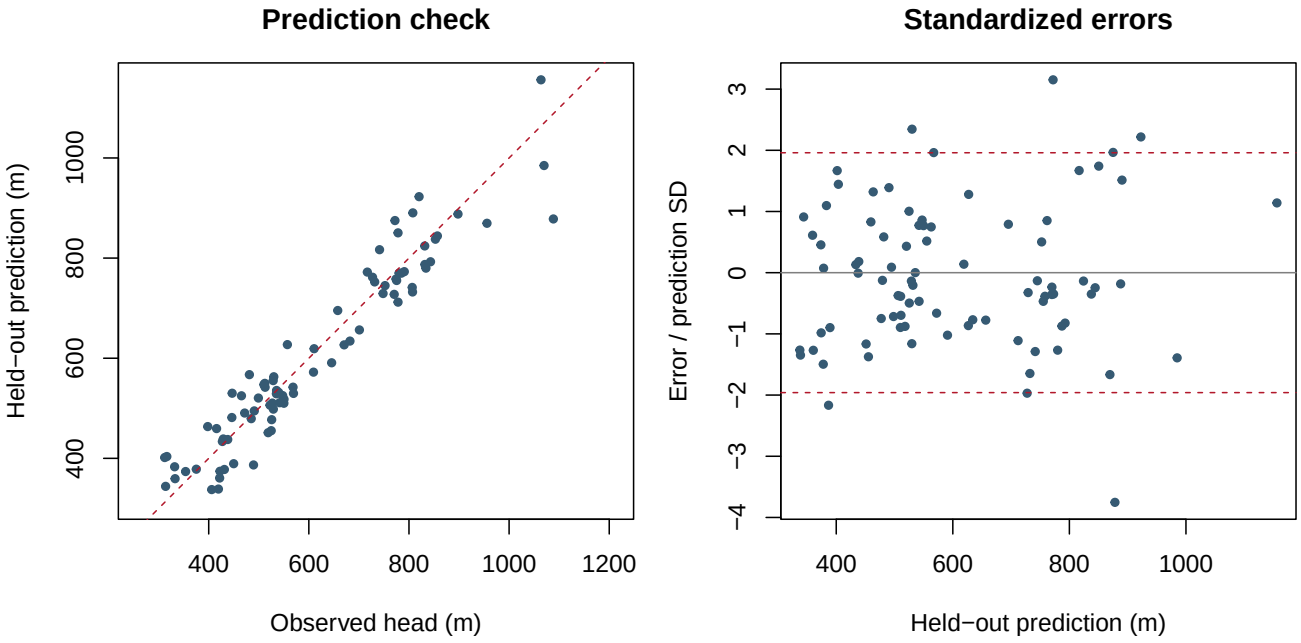
```
knitr::kable(cv_scores, digits = 2)
```

RMSE_m	MAE_m	Mean_error_m	MSSE	Coverage_95_percent
55.27	44.15	-5.12	1.35	90.59

The pooled RMSE is **55.27 m** and MAE is **44.15 m**. The mean error, defined as prediction minus observation, is **-5.12 m**, so the predictions are slightly low on average. Different folds select different models, which agrees with the close AIC scores in the full-data comparison.

The mean squared standardized error (MSSE) is **1.35**. A value near one would be consistent with errors on the scale of the predicted standard deviations. Here it is higher, and the approximate 95% intervals cover **90.6%** of the held-out heads. I therefore do not treat the uncertainty estimates as perfectly calibrated. They may be too narrow for some locations, and this check uses only 85 predictions from one split.

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
lim <- range(dat$head, cv_pred)
plot(dat$head, cv_pred, xlim = lim, ylim = lim, asp = 1,
     pch = 19, col = "#365A73", cex = 0.7,
     xlab = "Observed head (m)", ylab = "Held-out prediction (m)",
     main = "Prediction check")
abline(0, 1, col = "#B2182B", lty = 2)
plot(cv_pred, errors / sqrt(cv_var), pch = 19, col = "#365A73", cex = 0.7,
     xlab = "Held-out prediction (m)", ylab = "Error / prediction SD",
     main = "Standardized errors")
abline(h = 0, col = "grey50")
abline(h = c(-1.96, 1.96), col = "#B2182B", lty = 2)
```



Random folds can leave a nearby station in the training set. This check mainly describes prediction within the existing sampling network; prediction across large unsampled gaps may be harder. I have not repeated the partitions or used spatially separated folds here.

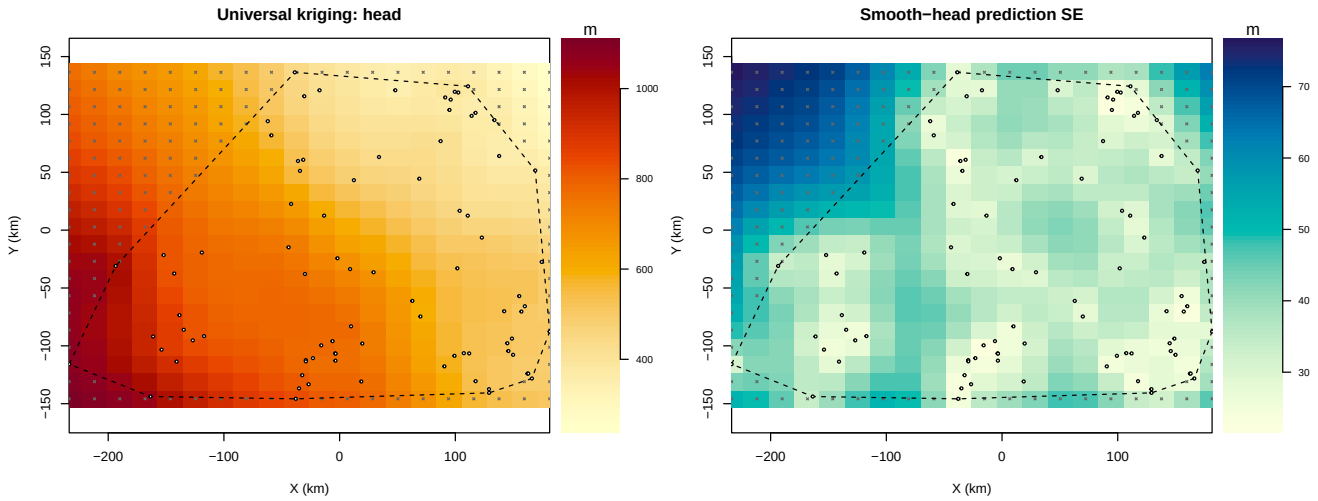
3.3 Final predictions on a 20 by 20 grid

I now use all 85 observations and the selected full-data model. I keep the same 20 by 20 grid definition as in Homework 1, using the observed coordinate ranges. I mark points outside the station convex hull so that the map does not hide where prediction extends beyond the sampled area.

```
grid_x <- seq(min(dat$x), max(dat$x), length.out = 20)
grid_y <- seq(min(dat$y), max(dat$y), length.out = 20)
grid <- expand.grid(x = grid_x, y = grid_y)
prediction <- predict_head(wolfcamp, grid, chosen, chosen_trend,
                           signal = TRUE)
grid$Head <- prediction$predict
stopifnot(all(is.finite(prediction$krige.var)),
          all(prediction$krige.var >= 0), all(is.finite(grid$Head)))
grid$SE <- sqrt(prediction$krige.var)
hull <- chull(dat$x, dat$y)
grid$Inside_hull <- sp::point.in.polygon(
  grid$x, grid$y, dat$x[hull], dat$y[hull]
) > 0
grid_summary <- data.frame(
  Locations = nrow(grid), Outside_hull = sum(!grid$Inside_hull),
  Min_head_m = min(grid$Head), Max_head_m = max(grid$Head),
  Min_SE_m = min(grid$SE), Max_SE_m = max(grid$SE)
)
knitr::kable(grid_summary, digits = 2)
```

Locations	Outside_hull	Min_head_m	Max_head_m	Min_SE_m	Max_SE_m
400	140	241.42	1107.12	21.78	76.5

```
draw_map <- function(values, title, colours, unit) {
  par(mar = c(4, 4, 3, 0.5))
  image(grid_x, grid_y, matrix(values, nrow = 20),
        col = colours, zlim = range(values), asp = 1, useRaster = TRUE,
        xlim = range(grid_x), ylim = range(grid_y),
        xlab = "X (km)", ylab = "Y (km)", main = title)
  points(grid$x[!grid$Inside_hull], grid$y[!grid$Inside_hull],
         pch = 4, col = "grey40", cex = 0.35)
  lines(dat$x[c(hull, hull[1])], dat$y[c(hull, hull[1])], lty = 2)
  points(dat$x, dat$y, pch = 21, bg = "white", cex = 0.5)
  par(mar = c(4, 0.2, 3, 3))
  levels <- seq(min(values), max(values), length.out = length(colours))
  image(c(0, 1), levels, matrix(rep(levels, each = 2), nrow = 2),
        col = colours, axes = FALSE, xlab = "", ylab = "")
  axis(4, las = 1, cex.axis = 0.7)
  mtext(unit, side = 3, cex = 0.8)
}
layout(matrix(1:4, nrow = 1), widths = c(1, 0.2, 1, 0.2))
draw_map(grid$Head, "Universal kriging: head",
         hcl.colors(100, "YlOrRd", rev = TRUE), "m")
draw_map(grid$SE, "Smooth-head prediction SE",
         hcl.colors(100, "YlGnBu", rev = TRUE), "m")
```




```
layout(1)
```

White circles are observed stations, the dashed line is their convex hull, and small crosses mark grid points outside it. Each panel has its own colour scale: the first shows head and the second its prediction standard error.

The predicted surface keeps the broad southwest-to-northeast decrease while allowing local changes around the observations. Predicted heads range from **241.4 to 1107.1 m**. Kriging with a trend is not restricted to the range of the observed heads, especially when predicting outside the sampled area.

There are **140 outside-hull points out of 400**. The smooth-head standard errors range from **21.8 to 76.5 m**, with larger values in poorly supported parts of the grid. Being inside the hull does not guarantee a nearby observation or an accurate model. The map also assumes that the fitted trend continues into areas where it is not directly checked.

For a new observed head at an unsampled location, the variance would include the nugget as well. With no transformation and no co-located observation, this adds `tau^2` to the smooth-head prediction variance while leaving the mean prediction unchanged. I keep this distinction in mind when interpreting the map and the held-out

prediction intervals.

4 My AI prompt and assessment

Prompt I used

> I am analysing the Wolfcamp data in R for Homework 3. I need to choose and justify a variogram model, then use kriging in the likelihood framework. Please check the data type, units, missing values and repeated locations. Examine the spatial trend and distribution before fitting the covariance. Show raw and residual variograms with distance groups and pair counts, and consider whether the patterns depend on direction. Compare a small set of reasonable models using Gaussian maximum likelihood, not just a visual fit to the variogram. Explain the parameter count, comparison rule, convergence checks, and what to do when models have similar support. Keep any needed trend or transformation consistent during fitting, prediction and validation. Make a 20 by 20 kriging map and an uncertainty map, and explain whether the target includes the nugget. If you check prediction accuracy, keep model selection inside the training data. Use simple R code and report only values actually calculated. Include the code, plots and clear explanations in an R Markdown report and compiled PDF, with cautious conclusions.

AI response recorded for this report

> I used the 85 Wolfcamp observations and confirmed the documented coordinate units of km and head units of m. A large-scale trend explains much of the raw spatial pattern. The detrended distribution is closer to Gaussian, so I retained the original scale and compared constant, linear and quadratic means with exponential, spherical and fixed-smoothness Matérn covariances.
>
> The quadratic exponential model has the smallest AIC, approximately 927.87. Under the stated rule of favouring fewer parameters within two AIC units, the selected model is linear-trend spherical, with AIC approximately 928.73. Its nugget is about 1236.24 m squared, partial sill 3328.95 m squared, and spherical range 122.95 km. This is a parsimonious choice among close models, not proof that the spherical covariance is uniquely correct.
>
> I used these ML estimates in universal kriging with the linear trend at both observed and prediction locations. The main map predicts the smooth head, while the five-fold validation predicts observed heads and includes the nugget in their variances. Repeating model selection within each training fold gives RMSE approximately 55.27 m. Covariance estimates are treated as fixed when computing kriging uncertainty, and 140 of the 400 grid points are outside the observation hull. These limits should be stated in the report.

My assessment

I find this answer useful because it separates the mean trend from the spatial covariance. It also uses likelihood estimation, which is what the assignment asks for. I can see both the strict AIC minimum and the reason for choosing a simpler model. This is clearer than saying that one model is best without showing its close alternatives.

I also need the distinction between a smooth underlying head and a new measured head. Their means can agree while their prediction variances differ. The validation results make me cautious about the uncertainty map, because the interval coverage is below the nominal 95%. I would not describe the model's standard errors as accounting for every source of uncertainty.

This response was generated during preparation of this report, in the same conversation and with access to the calculations. It is not an independent AI experiment. I include the prompt and response so that the AI contribution and the statistical claims are clear.

5 What I learned

My main choice is a **spherical variogram with a linear spatial trend**, fitted by maximum likelihood. The quadratic exponential model has a slightly lower AIC, but the linear spherical model gives a simpler explanation with similar support. I use that model for universal kriging rather than ignoring the trend.

I learned that choosing a variogram is more than matching a line to empirical points. The mean model, nugget, range, likelihood and numerical fitting all matter. Prediction accuracy is also different from uncertainty calibration: the held-out predictions are useful, but the approximate intervals miss more observations than their nominal level suggests.

The final maps depend on approximate Gaussian residuals, an isotropic stationary covariance after detrending, and estimated parameters treated as fixed. I would be most careful with locations far from stations and outside their hull. I have not shown that this covariance family is universally best, or that this one random validation split describes prediction across large unsampled areas.

My use of AI. I used AI assistance for the prompt, analysis plan, code and draft explanations, including help with the wording of my discussion. The results come from running the code. I remain responsible for checking the work and understanding the choices before submission.

My function references are the R help pages for `wolfcamp`, `variog`, `likfit`, `cov.spatial`, `krige.conv`, `krige.control` and `output.control` in `geoR`. In particular, I use the documented distinction between ML and REML, the covariance parameter conventions, and the `signal` option for prediction.

```
packages <- c("geoR", "sp", "knitr", "rmarkdown")
knitr::kable(data.frame(
  Component = c("R", packages),
  Version = c(as.character(getRversion()),
    vapply(packages, function(p) as.character(packageVersion(p)), character(1)))
), row.names = FALSE)
```

Component	Version
R	4.5.0
geoR	1.9.6
sp	2.2.0
knitr	1.50
rmarkdown	2.29